

BSK PG Trusted Third Party Demo

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	5
2.1 File List	5
3 Directory Documentation	7
3.1 apps Directory Reference	7
3.2 apps/client Directory Reference	7
3.3 apps/client/src/components Directory Reference	7
3.4 packages/crypto Directory Reference	7
3.5 apps/client/src/lib Directory Reference	8
3.6 packages Directory Reference	8
3.7 apps/client/src/routes Directory Reference	8
3.8 packages/rpc Directory Reference	8
3.9 apps/server Directory Reference	8
3.10 apps/client/src Directory Reference	8
3.11 apps/server/src Directory Reference	9
3.12 apps/ttp/src Directory Reference	9
3.13 packages/crypto/src Directory Reference	9
3.14 packages/rpc/src Directory Reference	9
3.15 apps/ttp Directory Reference	9
4 File Documentation	11
4.1 apps/client/src/api.ts File Reference	11
4.1.1 Function Documentation	11
4.1.1.1 encryptForTtp()	11
4.1.1.2 loadTtpPublicKey()	11
4.1.2 Variable Documentation	12
4.1.2.1 import	12
4.1.2.2 service	12
4.1.2.3 serviceBaseUrl	12
4.1.2.4 serviceQuery	12
4.1.2.5 ttp	12
4.1.2.6 ttpBaseUrl	12
4.1.2.7 ttpProtocol	12
4.1.2.8 ttpQuery	13
4.1.2.9 type	13
4.2 apps/client/src/components/evidence.tsx File Reference	13
4.2.1 Function Documentation	13
4.2.1.1 Evidence()	13
4.3 apps/client/src/components/utls.ts File Reference	13
4.3.1 Function Documentation	13

4.3.1.1 cn()	13
4.3.2 Variable Documentation	14
4.3.2.1 import	14
4.3.2.2 type	14
4.4 apps/client/src/lib/client-security-state.ts File Reference	14
4.4.1 Variable Documentation	14
4.4.1.1 ActiveSession	14
4.4.1.2 clientSecurityState	14
4.4.1.3 current	14
4.4.1.4 import	14
4.4.1.5 RegisteredUser	15
4.5 apps/client/src/lib/security-flow.ts File Reference	15
4.5.1 Function Documentation	15
4.5.1.1 activeSession()	15
4.5.1.2 createRogueUserCertificate()	15
4.5.1.3 loadRegisteredServer()	15
4.5.1.4 registerUserIdentity()	16
4.5.2 Variable Documentation	16
4.5.2.1 DemoFileCurrentResponse	16
4.5.2.2 DemoFileServiceRequest	16
4.5.2.3 DemoFileServiceResponse	16
4.5.2.4 DemoFileTransferInput	16
4.5.2.5 forge	17
4.5.2.6 import	17
4.5.2.7 RegisteredServer	17
4.5.2.8 securityFlow	17
4.5.2.9 ServerDemoFileCurrentResponse	17
4.6 apps/client/src/lib/useSecurityFlow.ts File Reference	17
4.6.1 Function Documentation	18
4.6.1.1 useSecurityFlow()	18
4.6.2 Variable Documentation	18
4.6.2.1 CLIENT_STATUS_KEY	18
4.6.2.2 import	18
4.7 apps/client/src/routes/index.tsx File Reference	18
4.7.1 Function Documentation	18
4.7.1.1 base64ByteLength()	18
4.7.1.2 base64ToText()	19
4.7.1.3 EncryptedPayloadView()	19
4.7.1.4 isTextFile()	19
4.7.1.5 readFileAsBase64()	19
4.7.1.6 SecurityFlowPage()	19
4.7.1.7 TransferredFilePreview()	19

4.7.2 Variable Documentation	19
4.7.2.1 FileServiceId	19
4.7.2.2 import	19
4.7.2.3 Route	20
4.7.2.4 type	20
4.8 apps/ttp/src/contract.ts File Reference	20
4.8.1 Function Documentation	21
4.8.1.1 serverAuthenticationPayload()	21
4.8.1.2 userAuthenticationPayload()	21
4.8.2 Variable Documentation	21
4.8.2.1 IdentityPublicKeys	21
4.8.2.2 import	21
4.8.2.3 RegisterIdentityInput	22
4.8.2.4 Role	22
4.8.2.5 ServerAuthenticationInput	22
4.8.2.6 ServerSessionKeyInput	22
4.8.2.7 UserAuthenticationInput	22
4.8.2.8 UserAuthenticationRequest	22
4.8.2.9 UserAuthRedirectInput	23
4.9 apps/server/src/protocol.ts File Reference	23
4.9.1 Function Documentation	23
4.9.1.1 closeLocalSession()	23
4.9.1.2 exchangeProtectedServiceData()	24
4.9.1.3 fetchServerSessionKey()	24
4.9.1.4 handleFileServiceRequest()	24
4.9.1.5 registerProtectedServer()	24
4.9.1.6 requestService()	25
4.9.2 Variable Documentation	25
4.9.2.1 DemoFileEmptyResponse	25
4.9.2.2 DemoFileResponse	25
4.9.2.3 DemoFileServiceRequest	25
4.9.2.4 DemoFileServiceResponse	26
4.9.2.5 forge	26
4.9.2.6 import	26
4.9.2.7 ttp	26
4.9.2.8 ttpBaseUrl	26
4.9.2.9 type	26
4.10 apps/ttp/src/protocol.ts File Reference	26
4.10.1 Function Documentation	27
4.10.1.1 authenticateServerCertificate()	27
4.10.1.2 authenticateUserForServer()	27
4.10.1.3 closeSession()	28

4.10.1.4 createSession()	28
4.10.1.5 registerIdentity()	28
4.10.1.6 requestUserAuthentication()	29
4.10.1.7 serverSessionKey()	29
4.10.1.8 ttpPublicKeyResponse()	29
4.10.2 Variable Documentation	29
4.10.2.1 authorityArtifactsSaved	29
4.10.2.2 CertificateClaim	29
4.10.2.3 forge	30
4.10.2.4 import	30
4.10.2.5 type	30
4.11 apps/server/src/rpc.ts File Reference	30
4.11.1 Function Documentation	30
4.11.1.1 reject()	30
4.11.2 Variable Documentation	30
4.11.2.1 import	30
4.11.2.2 ServiceRouter	31
4.11.2.3 type	31
4.12 apps/ttp/src/rpc.ts File Reference	31
4.12.1 Function Documentation	31
4.12.1.1 reject()	31
4.12.2 Variable Documentation	31
4.12.2.1 import	31
4.12.2.2 TtpRouter	31
4.12.2.3 type	32
4.13 apps/server/src/state.ts File Reference	32
4.13.1 Function Documentation	32
4.13.1.1 log()	32
4.13.1.2 readServiceServerStatus()	32
4.13.1.3 resetServiceServerState()	32
4.13.2 Variable Documentation	33
4.13.2.1 artifact	33
4.13.2.2 DemoFileMeta	33
4.13.2.3 DemoStoredFile	33
4.13.2.4 import	33
4.13.2.5 logger	33
4.13.2.6 ServiceServerState	33
4.13.2.7 ServiceServerStatus	34
4.13.2.8 state	34
4.14 apps/ttp/src/state.ts File Reference	34
4.14.1 Variable Documentation	34
4.14.1.1 artifact	34

4.14.1.2 ca	34
4.14.1.3 IdentityRecord	35
4.14.1.4 import	35
4.14.1.5 log	35
4.14.1.6 logger	35
4.14.1.7 PendingAuthRecord	35
4.14.1.8 pendingAuths	35
4.14.1.9 registeredIdentities	35
4.14.1.10 SessionRecord	35
4.14.1.11 sessions	36
4.14.1.12 sessionsByRequest	36
4.14.1.13 type	36
4.15 packages/crypto/src/certificates.ts File Reference	36
4.15.1 Function Documentation	36
4.15.1.1 createCertificateAuthority()	36
4.15.2 Variable Documentation	37
4.15.2.1 DAY_MS	37
4.15.2.2 forge	37
4.15.2.3 import	37
4.15.2.4 type	37
4.16 packages/crypto/src/crypto.ts File Reference	38
4.16.1 Function Documentation	38
4.16.1.1 encodeHybridPayloadBase64()	38
4.16.1.2 encryptRsaOaepSha256()	38
4.16.1.3 rsaOaepSha256()	38
4.16.2 Variable Documentation	39
4.16.2.1 AES_256_KEY_BYTES	39
4.16.2.2 AES_GCM_IV_BYTES	39
4.16.2.3 AES_GCM_TAG_BITS	39
4.16.2.4 aesGcm	39
4.16.2.5 forge	39
4.16.2.6 random	39
4.16.2.7 rsa	39
4.16.2.8 RSA_BITS	40
4.16.2.9 signedPayload	40
4.16.2.10 type	40
4.17 packages/crypto/src/types.ts File Reference	40
4.17.1 Variable Documentation	41
4.17.1.1 AesGcmPayload	41
4.17.1.2 CertificateAuthority	41
4.17.1.3 CertificateAuthorityOptions	41
4.17.1.4 forge	41

4.17.1.5 HybridEncryptedPayload	41
4.17.1.6 IdentityCertificateInput	42
4.17.1.7 RsaDecryptor	42
4.17.1.8 RsaEncryptor	42
4.17.1.9 RsaPair	42
4.17.1.10 SessionEncryptedPayload	42
4.17.1.11 SessionTicket	43
4.18 packages/rpc/src/client.ts File Reference	43
4.18.1 Variable Documentation	43
4.18.1.1 import	43
4.18.1.2 type	43
4.19 packages/rpc/src/log.ts File Reference	43
4.19.1 Function Documentation	44
4.19.1.1 artifactPath()	44
4.19.1.2 createSecurityLogger()	44
4.19.1.3 logDirectory()	44
4.19.1.4 logPath()	44
4.19.2 Variable Documentation	44
4.19.2.1 DEFAULT_LOG_DIR	44
4.19.2.2 import	44
4.19.2.3 pino	44
4.19.2.4 pretty	44
4.19.2.5 SecurityLogActor	44
4.19.2.6 SecurityLogLevel	45
4.20 packages/rpc/src/server.ts File Reference	45
4.20.1 Function Documentation	45
4.20.1.1 createRpcFetch()	45
4.20.2 Variable Documentation	45
4.20.2.1 import	45
4.20.2.2 type	45
Index	47

Chapter 1

Directory Hierarchy

1.1 Directories

apps	7
client	7
src	8
components	7
evidence.tsx	13
utils.ts	13
lib	8
client-security-state.ts	14
security-flow.ts	15
useSecurityFlow.ts	17
routes	8
index.tsx	18
api.ts	11
server	8
src	9
protocol.ts	23
rpc.ts	30
state.ts	32
ttp	9
src	9
contract.ts	20
protocol.ts	26
rpc.ts	31
state.ts	34
client	7
src	8
components	7
evidence.tsx	13
utils.ts	13
lib	8
client-security-state.ts	14
security-flow.ts	15
useSecurityFlow.ts	17
routes	8
index.tsx	18
api.ts	11

components	7
evidence.tsx	13
utils.ts	13
crypto	7
src	9
certificates.ts	36
crypto.ts	38
types.ts	40
lib	8
client-security-state.ts	14
security-flow.ts	15
useSecurityFlow.ts	17
packages	8
crypto	7
src	9
certificates.ts	36
crypto.ts	38
types.ts	40
rpc	8
src	9
client.ts	43
log.ts	43
server.ts	45
routes	8
index.tsx	18
rpc	8
src	9
client.ts	43
log.ts	43
server.ts	45
server	8
src	9
protocol.ts	23
rpc.ts	30
state.ts	32
src	8
components	7
evidence.tsx	13
utils.ts	13
lib	8
client-security-state.ts	14
security-flow.ts	15
useSecurityFlow.ts	17
routes	8
index.tsx	18
api.ts	11
src	9
protocol.ts	23
rpc.ts	30
state.ts	32
src	9
contract.ts	20
protocol.ts	26
rpc.ts	31

state.ts	34
src	9
certificates.ts	36
crypto.ts	38
types.ts	40
src	9
client.ts	43
log.ts	43
server.ts	45
ttp	9
src	9
contract.ts	20
protocol.ts	26
rpc.ts	31
state.ts	34

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

apps/client/src/ api.ts	11
apps/client/src/components/ evidence.tsx	13
apps/client/src/components/ utils.ts	13
apps/client/src/lib/ client-security-state.ts	14
apps/client/src/lib/ security-flow.ts	15
apps/client/src/lib/ useSecurityFlow.ts	17
apps/client/src/routes/ index.tsx	18
apps/server/src/ protocol.ts	23
apps/server/src/ rpc.ts	30
apps/server/src/ state.ts	32
apps/ttp/src/ contract.ts	20
apps/ttp/src/ protocol.ts	26
apps/ttp/src/ rpc.ts	31
apps/ttp/src/ state.ts	34
packages/crypto/src/ certificates.ts	36
packages/crypto/src/ crypto.ts	38
packages/crypto/src/ types.ts	40
packages/rpc/src/ client.ts	43
packages/rpc/src/ log.ts	43
packages/rpc/src/ server.ts	45

Chapter 3

Directory Documentation

3.1 apps Directory Reference

Directories

- directory [client](#)
- directory [server](#)
- directory [ttp](#)

3.2 apps/client Directory Reference

Directories

- directory [src](#)

3.3 apps/client/src/components Directory Reference

Files

- file [evidence.tsx](#)
- file [utils.ts](#)

3.4 packages/crypto Directory Reference

Directories

- directory [src](#)

3.5 apps/client/src/lib Directory Reference

Files

- file [client-security-state.ts](#)
- file [security-flow.ts](#)
- file [useSecurityFlow.ts](#)

3.6 packages Directory Reference

Directories

- directory [crypto](#)
- directory [rpc](#)

3.7 apps/client/src/routes Directory Reference

Files

- file [index.tsx](#)

3.8 packages/rpc Directory Reference

Directories

- directory [src](#)

3.9 apps/server Directory Reference

Directories

- directory [src](#)

3.10 apps/client/src Directory Reference

Directories

- directory [components](#)
- directory [lib](#)
- directory [routes](#)

Files

- file [api.ts](#)

3.11 apps/server/src Directory Reference

Files

- file [protocol.ts](#)
- file [rpc.ts](#)
- file [state.ts](#)

3.12 apps/ttp/src Directory Reference

Files

- file [contract.ts](#)
- file [protocol.ts](#)
- file [rpc.ts](#)
- file [state.ts](#)

3.13 packages/crypto/src Directory Reference

Files

- file [certificates.ts](#)
- file [crypto.ts](#)
- file [types.ts](#)

3.14 packages/rpc/src Directory Reference

Files

- file [client.ts](#)
- file [log.ts](#)
- file [server.ts](#)

3.15 apps/ttp Directory Reference

Directories

- directory [src](#)

Chapter 4

File Documentation

4.1 apps/client/src/api.ts File Reference

Functions

- function async [loadTtpPublicKey](#) ()
- function async [encryptForTtp](#) (plaintext:string)

Variables

- [import](#) { [rsa](#) } from "@bsk/crypto"
- [import type](#) { [ServiceRouter](#) } from "server"
- const [ttpBaseUrl](#) = import.meta.env.VITE_TTP_API_BASE_URL ?? "http://localhost:3001"
- const [serviceBaseUrl](#) = import.meta.env.VITE_SERVICE_API_BASE_URL ?? "http://localhost:3002"
- export const [ttp](#) = createRpcClient<[TtpRouter](#)>([ttpBaseUrl](#))
- export const [service](#) = createRpcClient<[ServiceRouter](#)>([serviceBaseUrl](#))
- export const [ttpQuery](#) = createTanstackQueryUtils([ttp](#))
- export const [serviceQuery](#) = createTanstackQueryUtils([service](#))
- export const [ttpProtocol](#)

4.1.1 Function Documentation

4.1.1.1 [encryptForTtp\(\)](#)

```
function async encryptForTtp (  
    plaintext:string)
```

4.1.1.2 [loadTtpPublicKey\(\)](#)

```
function async loadTtpPublicKey ()
```

4.1.2 Variable Documentation

4.1.2.1 import

```
import { rsa } from "@bsk/crypto"
```

4.1.2.2 service

```
export const service = createRpcClient<ServiceRouter>(serviceBaseUrl)
```

4.1.2.3 serviceBaseUrl

```
const serviceBaseUrl = import.meta.env.VITE_SERVICE_API_BASE_URL ?? "http://localhost:3002"
```

4.1.2.4 serviceQuery

```
export const serviceQuery = createTanstackQueryUtils(service)
```

4.1.2.5 ttp

```
export const ttp = createRpcClient<TtpRouter>(ttpBaseUrl)
```

4.1.2.6 ttpBaseUrl

```
const ttpBaseUrl = import.meta.env.VITE_TTP_API_BASE_URL ?? "http://localhost:3001"
```

4.1.2.7 ttpProtocol

```
export const ttpProtocol
```

Initial value:

```
    = {
  async registerUser(id: string, publicKey: IdentityPublicKeys) {
    return ttp.register({
      role: "user",
      encryptedId: await encryptForTtp(id),
      publicKey,
    });
  },

  requestUserAuthentication(requestId: string) {
    return ttp.auth.redirect({ requestId });
  },

  async authenticateUser(request: UserAuthenticationRequest) {
    return ttp.auth.user({
      encryptedAuthMaterial: await encryptForTtp(JSON.stringify(request)),
    });
  },

  closeSession(sessionId: string) {
    return ttp.session.close({ sessionId });
  },
}
```

4.1.2.8 ttpQuery

```
export const ttpQuery = createTanstackQueryUtils(ttp)
```

4.1.2.9 type

```
export type { ServiceRouter } from "server"
```

4.2 apps/client/src/components/evidence.tsx File Reference

Functions

- function export [Evidence](#) ({ label, value }: { label:string;value?:string })

4.2.1 Function Documentation

4.2.1.1 Evidence()

```
function export Evidence (  
    { label, value }: { label:string;value?:string })
```

4.3 apps/client/src/components/utils.ts File Reference

Functions

- function export [cn](#) (...inputs:ClassValue[])

Variables

- [import type](#) { ClassValue } from "clsx"
- [import](#) { clsx } from "clsx"

4.3.1 Function Documentation

4.3.1.1 cn()

```
function export cn (  
    ...inputs:ClassValue[])
```

4.3.2 Variable Documentation

4.3.2.1 import

```
import { clsx } from "clsx"
```

4.3.2.2 type

```
import type { ClassValue } from "clsx"
```

4.4 apps/client/src/lib/client-security-state.ts File Reference

Variables

- `import { hash, type RsaPair } from "@bsk/crypto"`
- `export type RegisteredUser`
- `type ActiveSession`
- `const current`
- `export const clientSecurityState`

4.4.1 Variable Documentation

4.4.1.1 [ActiveSession](#)

```
type ActiveSession
```

Initial value:

```
    = {  
      sessionId: string;  
      userSessionKey: string;  
    }
```

4.4.1.2 [clientSecurityState](#)

```
export const clientSecurityState
```

4.4.1.3 [current](#)

```
const current
```

4.4.1.4 import

```
import { hash, type RsaPair } from "@bsk/crypto"
```

4.4.1.5 RegisteredUser

```
export type RegisteredUser
```

Initial value:

```
    = {  
  id: string;  
  authKeyPair: RsaPair;  
  exchangeKeyPair: RsaPair;  
  certificatePem: string;  
}
```

4.5 apps/client/src/lib/security-flow.ts File Reference

Functions

- function async [registerUserIdentity](#) ()
- function async [createRogueUserCertificate](#) (user:RegisteredUser)
- function [activeSession](#) ()
- function async [loadRegisteredServer](#) ()

Variables

- [import](#)
- [import](#) forge from node [forge](#)
- [type](#) [RegisteredServer](#)
- [type](#) [DemoFileServiceRequest](#)
- [export type](#) [DemoFileTransferInput](#)
- [export type](#) [DemoFileCurrentResponse](#)
- [type](#) [ServerDemoFileCurrentResponse](#) = Omit<[DemoFileCurrentResponse](#), "encryptedPayload">
- [export type](#) [DemoFileServiceResponse](#)
- [export const](#) [securityFlow](#)

4.5.1 Function Documentation

4.5.1.1 activeSession()

```
function activeSession ()
```

4.5.1.2 createRogueUserCertificate()

```
function async createRogueUserCertificate (  
  user:RegisteredUser)
```

4.5.1.3 loadRegisteredServer()

```
function async loadRegisteredServer ()
```

4.5.1.4 registerUserIdentity()

```
function async registerUserIdentity ()
```

4.5.2 Variable Documentation

4.5.2.1 DemoFileCurrentResponse

```
export type DemoFileCurrentResponse
```

Initial value:

```

= {
  kind: "file.current";
  name: string;
  mimeType: string;
  size: number;
  contentBase64: string;
  storedAt: string;
  encryptedPayload: SessionEncryptedPayload;
}
```

4.5.2.2 DemoFileServiceRequest

```
type DemoFileServiceRequest
```

Initial value:

```

=
| {
|   kind: "file.upload";
|   name: string;
|   mimeType: string;
|   contentBase64: string;
| }
| {
|   kind: "file.view";
| }
```

4.5.2.3 DemoFileServiceResponse

```
export type DemoFileServiceResponse
```

Initial value:

```

=
| DemoFileCurrentResponse
| {
|   kind: "file.empty";
| }
```

4.5.2.4 DemoFileTransferInput

```
export type DemoFileTransferInput
```

Initial value:

```

= {
  name: string;
  mimeType: string;
  contentBase64: string;
}
```


4.5.2.5 forge

```
import forge from node forge
```

4.5.2.6 import

```
import
```

Initial value:

```
{
  aesGcm,
  createCertificateAuthority,
  hash,
  issueIdentityCertificate,
  random,
  rsa,
  type SessionEncryptedPayload,
  type SessionTicket,
} from "@bsk/crypto"
```

4.5.2.7 RegisteredServer

```
type RegisteredServer
```

Initial value:

```
      = Awaited<ReturnType<typeof service.state>> & {
  serverId: string;
  certificatePem: string;
}
```

4.5.2.8 securityFlow

```
export const securityFlow
```

4.5.2.9 ServerDemoFileCurrentResponse

```
type ServerDemoFileCurrentResponse = Omit<DemoFileCurrentResponse, "encryptedPayload">
```

4.6 apps/client/src/lib/useSecurityFlow.ts File Reference

Functions

- function export `useSecurityFlow` ()

Variables

- `import` { useMutation, useQuery, useQueryClient } from "@tanstack/react-query"
- `const CLIENT_STATUS_KEY` = ["client-status"] as const

4.6.1 Function Documentation

4.6.1.1 useSecurityFlow()

```
function export useSecurityFlow ()
```

4.6.2 Variable Documentation

4.6.2.1 CLIENT_STATUS_KEY

```
const CLIENT_STATUS_KEY = ["client-status"] as const
```

4.6.2.2 import

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query"
```

4.7 apps/client/src/routes/index.tsx File Reference

Functions

- function [SecurityFlowPage](#) ()
- function [TransferredFilePreview](#) ({ file }: { file: DemoFileCurrentResponse })
- function [EncryptedPayloadView](#) ({ file }: { file: DemoFileCurrentResponse })
- function [readFileAsBase64](#) (file: File)
- function [isTextFile](#) (file: DemoFileCurrentResponse)
- function [base64ByteLength](#) (value: string)
- function [base64ToText](#) (value: string)

Variables

- [import](#) { createFileRoute } from "@tanstack/react-router"
- [import type](#)
- [type](#) FileServiceId = "upload" | "view"
- export const [Route](#)

4.7.1 Function Documentation

4.7.1.1 base64ByteLength()

```
function base64ByteLength (  
    value: string
```

4.7.1.2 base64ToText()

```
function base64ToText (
  value:string)
```

4.7.1.3 EncryptedPayloadView()

```
function EncryptedPayloadView (
  { file }: { file:DemoFileCurrentResponse })
```

4.7.1.4 isTextFile()

```
function isTextFile (
  file:DemoFileCurrentResponse)
```

4.7.1.5 readFileAsBase64()

```
function readFileAsBase64 (
  file:File)
```

4.7.1.6 SecurityFlowPage()

```
function SecurityFlowPage ()
```

4.7.1.7 TransferredFilePreview()

```
function TransferredFilePreview (
  { file }: { file:DemoFileCurrentResponse })
```

4.7.2 Variable Documentation

4.7.2.1 FileServiceId

```
type FileServiceId = "upload" | "view"
```

4.7.2.2 import

```
import { createFileRoute } from "@tanstack/react-router"
```

4.7.2.3 Route

```
export const Route
```

Initial value:

```
    = createFileRoute("/") ({
  component: SecurityFlowPage,
})
```

4.7.2.4 type

```
import type
```

Initial value:

```
{
  DemoFileCurrentResponse,
  DemoFileServiceResponse,
  DemoFileTransferInput,
} from "@/lib/security-flow"
```

4.8 apps/ttp/src/contract.ts File Reference

Functions

- function export `serverAuthenticationPayload` (input:{ serverId:string;certificatePem:string;requestId:↔string;userId:string;})
Builds the payload signed by the Server.
- function export `userAuthenticationPayload` (input:{ userId:string;userCertificatePem:string;serverId:↔string;serverCertificatePem:string;requestId:string;})
Builds the canonical payload signed by the User.

Variables

- import { hash, signedPayload } from "@bsk/crypto"
- export type Role = "user" | "server"
- export type IdentityPublicKeys
- export type RegisterIdentityInput
- export type ServerAuthenticationInput
- export type UserAuthenticationRequest
- export type UserAuthenticationInput
- export type ServerSessionKeyInput
- export type UserAuthRedirectInput

4.8.1 Function Documentation

4.8.1.1 serverAuthenticationPayload()

```
function export serverAuthenticationPayload (  
    input:{ serverId:string;certificatePem:string;requestId:string;userId:string;})
```

Builds the payload signed by the Server.

Parameters

<i>input</i>	Server identity, User identity, request id and certificate.
--------------	---

Returns

JSON string used for RSA signature verification.

4.8.1.2 userAuthenticationPayload()

```
function export userAuthenticationPayload (  
    input:{ userId:string;userCertificatePem:string;serverId:string;serverCertificatePem:string;requestId:string;})
```

Builds the canonical payload signed by the User.

Parameters

<i>input</i>	User, Server, request id and certificate material.
--------------	--

Returns

Stable JSON string used before session-key issuance.

4.8.2 Variable Documentation

4.8.2.1 IdentityPublicKeys

```
export type IdentityPublicKeys
```

Initial value:

```
    = {  
    authPublicKeyPem: string;  
    exchangePublicKeyPem: string;  
    }
```

4.8.2.2 import

```
import { hash, signedPayload } from "@bsk/crypto"
```

4.8.2.3 RegisterIdentityInput

```
export type RegisterIdentityInput
```

Initial value:

```
    = {  
  role: Role;  
  encryptedId: string;  
  publicKeys: IdentityPublicKeys;  
}
```

4.8.2.4 Role

```
export type Role = "user" | "server"
```

4.8.2.5 ServerAuthenticationInput

```
export type ServerAuthenticationInput
```

Initial value:

```
    = {  
  serverId: string;  
  certificatePem: string;  
  requestId: string;  
  userId: string;  
  signature: string;  
}
```

4.8.2.6 ServerSessionKeyInput

```
export type ServerSessionKeyInput
```

Initial value:

```
    = {  
  requestId: string;  
}
```

4.8.2.7 UserAuthenticationInput

```
export type UserAuthenticationInput
```

Initial value:

```
    = {  
  encryptedAuthMaterial: string;  
}
```

4.8.2.8 UserAuthenticationRequest

```
export type UserAuthenticationRequest
```

Initial value:

```
    = {  
  userId: string;  
  userCertificatePem: string;  
  serverId: string;  
  serverCertificatePem: string;  
  requestId: string;  
  signature: string;  
}
```

4.8.2.9 UserAuthRedirectInput

```
export type UserAuthRedirectInput
```

Initial value:

```
    = {
  requestId: string;
}
```

4.9 apps/server/src/protocol.ts File Reference

Functions

- function async [registerProtectedServer](#) ()
Registers the protected Server with the TTP.
- function async [requestService](#) (input:{ userId:string })
Starts a protected service request by authenticating the Server to the TTP.
- function async [fetchServerSessionKey](#) (input:{ requestId:string })
Fetches and decrypts the Server session ticket.
- function export [closeLocalSession](#) ()
Clears the Server's local session state after service completion.
- function export [exchangeProtectedServiceData](#) (payload:SessionEncryptedPayload)
Handles one encrypted protected-service request.
- function [handleFileServiceRequest](#) (request:DemoFileServiceRequest)

Variables

- [import](#)
- [import](#) forge from node [forge](#)
- [import type](#) { [TtpRouter](#) } from "ttp"
- const [ttpBaseUrl](#) = process.env.TTP_API_BASE_URL ?? "http://localhost:3001"
- const [ttp](#) = createRpcClient<[TtpRouter](#)>([ttpBaseUrl](#))
- [type](#) [DemoFileServiceRequest](#)
- [type](#) [DemoFileResponse](#)
- [type](#) [DemoFileEmptyResponse](#)
- [type](#) [DemoFileServiceResponse](#) = [DemoFileResponse](#) | [DemoFileEmptyResponse](#)

4.9.1 Function Documentation

4.9.1.1 closeLocalSession()

```
function export { closeLocalSession ()
```

Clears the Server's local session state after service completion.

Returns

Previous session id and updated Server status.

4.9.1.2 exchangeProtectedServiceData()

```
function export exchangeProtectedServiceData (  
    payload:SessionEncryptedPayload)
```

Handles one encrypted protected-service request.

Parameters

<i>payload</i>	AES-GCM payload bound to the active session id.
----------------	---

Returns

Encrypted service response using the same session key.

4.9.1.3 fetchServerSessionKey()

```
function async fetchServerSessionKey (  
    input:{ requestId:string })
```

Fetches and decrypts the Server session ticket.

Parameters

<i>input</i>	Service request id whose User authentication has completed.
--------------	---

Returns

Updated Server status with an active session.

4.9.1.4 handleFileServiceRequest()

```
function handleFileServiceRequest (  
    request:DemoFileServiceRequest)
```

4.9.1.5 registerProtectedServer()

```
function async registerProtectedServer ()
```

Registers the protected Server with the TTP.

Returns

Current Server status including its issued certificate.

4.9.1.6 requestService()

```
function async requestService (
    input:{ userId:string })
```

Starts a protected service request by authenticating the Server to the TTP.

Parameters

<i>input</i>	User id requesting the service.
--------------	---------------------------------

Returns

Request id used for the following User-authentication step.

4.9.2 Variable Documentation

4.9.2.1 DemoFileEmptyResponse

```
type DemoFileEmptyResponse
```

Initial value:

```
= {
  kind: "file.empty";
}
```

4.9.2.2 DemoFileResponse

```
type DemoFileResponse
```

Initial value:

```
= {
  kind: "file.current";
  name: string;
  mimeType: string;
  size: number;
  contentBase64: string;
  storedAt: string;
}
```

4.9.2.3 DemoFileServiceRequest

```
type DemoFileServiceRequest
```

Initial value:

```
=
| {
|   kind: "file.upload";
|   name: string;
|   mimeType: string;
|   contentBase64: string;
| }
| {
|   kind: "file.view";
| }
```

4.9.2.4 DemoFileServiceResponse

```
type DemoFileServiceResponse = DemoFileResponse | DemoFileEmptyResponse
```

4.9.2.5 forge

```
import forge from node forge
```

4.9.2.6 import

```
import
```

Initial value:

```
{
  aesGcm,
  hash,
  random,
  rsa,
  type SessionEncryptedPayload,
  type SessionTicket,
} from "@bsk/crypto"
```

4.9.2.7 ttp

```
const ttp = createRpcClient<TtpRouter>(ttpBaseUrl)
```

4.9.2.8 ttpBaseUrl

```
const ttpBaseUrl = process.env.TTP_API_BASE_URL ?? "http://localhost:3001"
```

4.9.2.9 type

```
import type { TtpRouter } from "ttp"
```

4.10 apps/ttp/src/protocol.ts File Reference

Functions

- function async [ttpPublicKeyResponse](#) ()
- function async [registerIdentity](#) (input:RegisterIdentityInput)
Registers a User or Server and issues its identity certificate.
- function export [authenticateServerCertificate](#) (input:ServerAuthenticationInput)
Validates Server authentication and creates a pending User-auth request.
- function export [requestUserAuthentication](#) (input:UserAuthRedirectInput)
Returns the User-authentication redirect after Server validation.
- function async [authenticateUserForServer](#) (input:UserAuthenticationInput)
Validates User authentication and issues encrypted session tickets.
- function export [serverSessionKey](#) (input:ServerSessionKeyInput)
Returns the Server copy of the encrypted session ticket.
- function export [closeSession](#) (sessionId:string)
Marks a negotiated session as closed in the TTP state.
- function [createSession](#) (requestId:string, userId:string, serverId:string)

Variables

- `import`
- `import` `forge` from node `forge`
- `import type` { `IdentityRecord`, `SessionRecord` } from `"./state"`
- `let` `authorityArtifactsSaved` = `false`
- `type` `CertificateClaim`

4.10.1 Function Documentation

4.10.1.1 `authenticateServerCertificate()`

```
function export authenticateServerCertificate (  
    input:ServerAuthenticationInput)
```

Validates Server authentication and creates a pending User-auth request.

Parameters

<i>input</i>	Server certificate, request id, User id and Server signature.
--------------	---

Returns

Server-authentication confirmation for the request.

4.10.1.2 `authenticateUserForServer()`

```
function async authenticateUserForServer (  
    input:UserAuthenticationInput)
```

Validates User authentication and issues encrypted session tickets.

Parameters

<i>input</i>	User authentication material encrypted to the TTP.
--------------	--

Returns

The validated request and encrypted User session ticket.

4.10.1.3 closeSession()

```
function export closeSession (  
    sessionId:string)
```

Marks a negotiated session as closed in the TTP state.

Parameters

<i>sessionId</i>	Session identifier to close.
------------------	------------------------------

Returns

Close confirmation with timestamp.

4.10.1.4 createSession()

```
function createSession (  
    requestId:string,  
    userId:string,  
    serverId:string)
```

4.10.1.5 registerIdentity()

```
function async registerIdentity (  
    input:RegisterIdentityInput)
```

Registers a User or Server and issues its identity certificate.

Parameters

<i>input</i>	Encrypted subject id, role and public keys submitted by the identity.
--------------	---

Returns

Registered subject id, certificate and issuance timestamp.

4.10.1.6 requestUserAuthentication()

```
function export requestUserAuthentication (
    input:UserAuthRedirectInput)
```

Returns the User-authentication redirect after Server validation.

Parameters

<i>input</i>	Service request id.
--------------	---------------------

Returns

Redirect metadata telling the Client to submit User authentication.

4.10.1.7 serverSessionKey()

```
function export serverSessionKey (
    input:ServerSessionKeyInput)
```

Returns the Server copy of the encrypted session ticket.

Parameters

<i>input</i>	Completed service request id.
--------------	-------------------------------

Returns

Session id and ticket encrypted with the Server exchange key.

4.10.1.8 ttpPublicKeyResponse()

```
function async ttpPublicKeyResponse ()
```

4.10.2 Variable Documentation

4.10.2.1 authorityArtifactsSaved

```
let authorityArtifactsSaved = false
```

4.10.2.2 CertificateClaim

```
type CertificateClaim
```

Initial value:

```
    = Pick<IdentityRecord, "role" | "subjectId"> & {
    certificatePem: string;
}
```

4.10.2.3 forge

```
import forge from node forge
```

4.10.2.4 import

```
import
```

Initial value:

```
{
  issueIdentityCertificate,
  random,
  rsa,
  verifyCertificateSignedBy,
  type SessionTicket,
} from "@bsk/crypto"
```

4.10.2.5 type

```
export type { IdentityRecord, SessionRecord } from "./state"
```

4.11 apps/server/src/rpc.ts File Reference

Functions

- function `reject` (code:"BAD_REQUEST"|"UNAUTHORIZED", error:unknown)

Variables

- `import type { SessionEncryptedPayload } from "@bsk/crypto"`
- `import { ORPCError, os, type } from "@orpc/server"`
- `export type ServiceRouter = typeof serviceRouter`

4.11.1 Function Documentation

4.11.1.1 reject()

```
function reject (
  code:"BAD_REQUEST"|"UNAUTHORIZED",
  error:unknown)
```

4.11.2 Variable Documentation

4.11.2.1 import

```
import { ORPCError, os, type } from "@orpc/server"
```

4.11.2.2 ServiceRouter

```
export type ServiceRouter = typeof serviceRouter
```

4.11.2.3 type

```
import type { SessionEncryptedPayload } from "@bsk/crypto"
```

4.12 apps/ttp/src/rpc.ts File Reference

Functions

- function `reject` (code:"BAD_REQUEST"|"UNAUTHORIZED"|"NOT_FOUND", error:unknown)

Variables

- `import { RSA_BITS } from "@bsk/crypto"`
- `import type`
- `export type TtpRouter = typeof ttpRouter`

4.12.1 Function Documentation

4.12.1.1 reject()

```
function reject (
    code:"BAD_REQUEST"|"UNAUTHORIZED"|"NOT_FOUND",
    error:unknown)
```

4.12.2 Variable Documentation

4.12.2.1 import

```
import { RSA_BITS } from "@bsk/crypto"
```

4.12.2.2 TtpRouter

```
export type TtpRouter = typeof ttpRouter
```

4.12.2.3 type

```
import type
```

Initial value:

```
{
  RegisterIdentityInput,
  ServerAuthenticationInput,
  ServerSessionKeyInput,
  UserAuthenticationInput,
  UserAuthRedirectInput,
} from "../contract"
```

4.13 apps/server/src/state.ts File Reference

Functions

- function export [log](#) (event:string, details:string, level:SecurityLogLevel="info")
- function export [resetServiceServerState](#) ()
- function export [readServiceServerStatus](#) ()

Variables

- [import { type RsaPair }](#) from "@bsk/crypto"
- [type ServiceServerState](#)
- export interface [ServiceServerStatus](#)
- export const [state](#)
- export const export [type DemoFileMeta](#)
- export [type DemoStoredFile](#)
- const [logger](#) = [createSecurityLogger](#)(["application.log", "server.log"])
- export const [artifact](#) = logger.artifact

4.13.1 Function Documentation

4.13.1.1 log()

```
function export log (
    event:string,
    details:string,
    level:SecurityLogLevel = "info")
```

4.13.1.2 readServiceServerStatus()

```
function export readServiceServerStatus ()
```

4.13.1.3 resetServiceServerState()

```
function export resetServiceServerState ()
```


4.13.2 Variable Documentation

4.13.2.1 artifact

```
export const artifact = logger.artifact
```

4.13.2.2 DemoFileMeta

```
export const export type DemoFileMeta
```

Initial value:

```
    = {  
      name: string;  
      size: number;  
      mimeType: string;  
    }
```

4.13.2.3 DemoStoredFile

```
export type DemoStoredFile
```

Initial value:

```
    = DemoFileMeta & {  
      contentBase64: string;  
      storedAt: string;  
    }
```

4.13.2.4 import

```
import { type RsaPair } from "@bsk/crypto"
```

4.13.2.5 logger

```
const logger = createSecurityLogger(["application.log", "server.log"])
```

4.13.2.6 ServiceServerState

```
type ServiceServerState
```

Initial value:

```
    = {  
      serverId?: string;  
      authKeyPair?: RsaPair;  
      exchangeKeyPair?: RsaPair;  
      certificatePem?: string;  
      sessionId?: string;  
      sessionKey?: string;  
      serviceExchanged?: boolean;  
      latestDemoFile?: DemoStoredFile;  
      lastServiceEvent?: { event: string; details: string };  
      pendingRequests: Map<string, { userId: string; createdAt: string }>;  
    }
```

4.13.2.7 ServiceServerStatus

```
export interface ServiceServerStatus
```

Initial value:

```
{  
  registered: boolean
```

4.13.2.8 state

```
export const state
```

4.14 apps/ttp/src/state.ts File Reference

Variables

- `import { createCertificateAuthority } from "@bsk/crypto"`
- `import type { IdentityPublicKeys, Role } from "../contract"`
- `export interface IdentityRecord`
- `export interface SessionRecord`
- `export interface PendingAuthRecord`
- `export const ca`
- `export const registeredIdentities = new Map<string, IdentityRecord>()`
- `export const sessions = new Map<string, SessionRecord>()`
- `export const sessionsByRequest = new Map<string, string>()`
- `export const pendingAuths = new Map<string, PendingAuthRecord>()`
- `const logger = createSecurityLogger(["application.log", "ttp.log"])`
- `export const log = logger.log`
- `export const artifact = logger.artifact`

4.14.1 Variable Documentation

4.14.1.1 artifact

```
export const artifact = logger.artifact
```

4.14.1.2 ca

```
export const ca
```

Initial value:

```
    = await createCertificateAuthority({  
      commonName: "BSK PG Trusted Third Party",  
      organization: "BSK PG Demo",  
    })
```

4.14.1.3 IdentityRecord

```
export interface IdentityRecord
```

Initial value:

```
{  
  role: Role
```

4.14.1.4 import

```
import { createCertificateAuthority } from "@bsk/crypto"
```

4.14.1.5 log

```
export const log = logger.log
```

4.14.1.6 logger

```
const logger = createSecurityLogger(["application.log", "ttp.log"])
```

4.14.1.7 PendingAuthRecord

```
export interface PendingAuthRecord
```

Initial value:

```
{  
  requestId: string
```

4.14.1.8 pendingAuths

```
export const pendingAuths = new Map<string, PendingAuthRecord>()
```

4.14.1.9 registeredIdentities

```
export const registeredIdentities = new Map<string, IdentityRecord>()
```

4.14.1.10 SessionRecord

```
export interface SessionRecord
```

Initial value:

```
{  
  sessionId: string
```

4.14.1.11 sessions

```
export const sessions = new Map<string, SessionRecord>()
```

4.14.1.12 sessionsByRequest

```
export const sessionsByRequest = new Map<string, string>()
```

4.14.1.13 type

```
import type { IdentityPublicKeys, Role } from "../contract"
```

4.15 packages/crypto/src/certificates.ts File Reference

Functions

- function async [createCertificateAuthority](#) (options:CertificateAuthorityOptions,)

Creates the self-signed TTP certificate authority.

Variables

- [import](#) [forge](#) from node [forge](#)
- [import](#) { [random](#), [rsa](#) } from "../crypto"
- [import](#) type
- const [DAY_MS](#) = 24 * 60 * 60 * 1000

4.15.1 Function Documentation

4.15.1.1 createCertificateAuthority()

```
function async createCertificateAuthority (
    options:CertificateAuthorityOptions)
```

Creates the self-signed TTP certificate authority.

Parameters

<i>options</i>	Certificate subject and validity settings.
----------------	--

Returns

PEM keys and the forge certificate object used for signing identity certificates. Issues a TTP-signed X.509 identity certificate.

Parameters

<i>input</i>	Registered identity, exchange public key and issuing authority.
--------------	---

Returns

PEM-encoded X.509 certificate for the User or Server exchange key. Verifies that a certificate chains to the trusted TTP authority.

Parameters

<i>authority</i>	Trusted TTP certificate authority.
<i>certificatePem</i>	PEM-encoded certificate to validate.

Returns

The validated certificate common name.

4.15.2 Variable Documentation

4.15.2.1 DAY_MS

```
const DAY_MS = 24 * 60 * 60 * 1000
```

4.15.2.2 forge

```
import forge from node forge
```

4.15.2.3 import

```
import { random, rsa } from "../crypto"
```

4.15.2.4 type

```
import type
```

Initial value:

```
{  
  CertificateAuthority,  
  CertificateAuthorityOptions,  
  IdentityCertificateInput,  
} from "../types"
```

4.16 packages/crypto/src/crypto.ts File Reference

Functions

- function `rsaOaepSha256` ()
- function `encryptRsaOaepSha256` (publicKey:forge.pki.rsa.PublicKey, plaintext:string)
- function `encodeHybridPayloadBase64` (payload:HybridEncryptedPayload)

Variables

- `import` forge from node `forge`
- `import` type
- `export const` `RSA_BITS` = 4096
RSA modulus size required by the project specification.
- `const` `AES_GCM_TAG_BITS` = 128
AES-GCM tag length used for authenticated session payloads.
- `const` `AES_GCM_IV_BYTES` = 12
- `const` `AES_256_KEY_BYTES` = 32
AES-256 session key length in bytes.
- `export const` `random`
- `export const` `signedPayload`
- `export const` `aesGcm`
- `export const` `rsa`

4.16.1 Function Documentation

4.16.1.1 `encodeHybridPayloadBase64()`

```
function encodeHybridPayloadBase64 (
    payload:HybridEncryptedPayload)
```

Creates SHA-256 identifiers and short fingerprints for certificates or UI evidence.

4.16.1.2 `encryptRsaOaepSha256()`

```
function encryptRsaOaepSha256 (
    publicKey:forge.pki.rsa. PublicKey,
    plaintext:string)
```

4.16.1.3 `rsaOaepSha256()`

```
function rsaOaepSha256 ()
```

4.16.2 Variable Documentation

4.16.2.1 AES_256_KEY_BYTES

```
const AES_256_KEY_BYTES = 32
```

AES-256 session key length in bytes.

4.16.2.2 AES_GCM_IV_BYTES

```
const AES_GCM_IV_BYTES = 12
```

4.16.2.3 AES_GCM_TAG_BITS

```
const AES_GCM_TAG_BITS = 128
```

AES-GCM tag length used for authenticated session payloads.

4.16.2.4 aesGcm

```
export const aesGcm
```

4.16.2.5 forge

```
import forge from node forge
```

4.16.2.6 random

```
export const random
```

Initial value:

```
    = {  
    hex(bytes: number): string {  
        const value = forge.random.getBytesSync(bytes);  
        return forge.util.bytesToHex(value);  
    },  
    uuid() {  
        return crypto.randomUUID();  
    },  
    sessionKey(): string {  
        const key = forge.random.getBytesSync(AES_256_KEY_BYTES);  
        return forge.util.encode64(key);  
    },  
    }
```

4.16.2.7 rsa

```
export const rsa
```

4.16.2.8 RSA_BITS

```
export const RSA_BITS = 4096
```

RSA modulus size required by the project specification.

4.16.2.9 signedPayload

```
export const signedPayload
```

Initial value:

```
    = {
      from(fields: Record<string, string>): string {
        const sortedFields = Object.fromEntries(
          Object.entries(fields).sort(([left], [right]) => left.localeCompare(right)),
        );
        return JSON.stringify(sortedFields);
      },
    }
```

4.16.2.10 type

```
import type
```

Initial value:

```
{
  AesGcmPayload,
  HybridEncryptedPayload,
  SessionEncryptedPayload,
  RsaDecryptor,
  RsaEncryptor,
  RsaPair,
} from "../types"
```

4.17 packages/crypto/src/types.ts File Reference

Variables

- `import type` `forge` from node `forge`
- `export type` `RsaPair`
- `export interface` `RsaEncryptor`
- `verify`(plaintext:string, signatureBase64:string) `export interface` `RsaDecryptor`
Verifies a base64 RSA/SHA-256 signature for the given plaintext.
- `sign`(plaintext:string) `export type` `AesGcmPayload`
Signs the given plaintext with RSA/SHA-256 and returns a base64 signature.
- `export type` `SessionEncryptedPayload`
- `export type` `SessionTicket`
- `export type` `HybridEncryptedPayload`
- `export type` `CertificateAuthorityOptions`
- `export type` `CertificateAuthority`
- `export type` `IdentityCertificateInput`

4.17.1 Variable Documentation

4.17.1.1 AesGcmPayload

```
sign (plaintext: string) export type AesGcmPayload
```

Initial value:

```
    = {  
      iv: string;  
      ciphertext: string;  
      authTag: string;  
    }
```

Signs the given plaintext with RSA/SHA-256 and returns a base64 signature.

4.17.1.2 CertificateAuthority

```
export type CertificateAuthority
```

Initial value:

```
    = {  
      privateKey: forge.pki.rsa.PrivateKey;  
      certificate: forge.pki.Certificate;  
      privateKeyPem: string;  
      publicKeyPem: string;  
      certificatePem: string;  
    }
```

4.17.1.3 CertificateAuthorityOptions

```
export type CertificateAuthorityOptions
```

Initial value:

```
    = {  
      commonName: string;  
      organization: string;  
      validDays?: number;  
    }
```

4.17.1.4 forge

```
import type forge from node forge
```

4.17.1.5 HybridEncryptedPayload

```
export type HybridEncryptedPayload
```

Initial value:

```
    = AesGcmPayload & {  
      encryptedKey: string;  
    }
```

4.17.1.6 IdentityCertificateInput

```
export type IdentityCertificateInput
```

Initial value:

```
    = {  
    authority: CertificateAuthority;  
    role: string;  
    subjectId: string;  
    publicKeyPem: string;  
    organization?: string;  
    validDays?: number;  
    }
```

4.17.1.7 RsaDecryptor

```
verify (plaintext: string, signatureBase64: string) export interface RsaDecryptor
```

Initial value:

```
{  
    decrypt(payloadBase64: string): string  
}
```

Verifies a base64 RSA/SHA-256 signature for the given plaintext.

4.17.1.8 RsaEncryptor

```
export interface RsaEncryptor
```

Initial value:

```
{  
    encrypt(plaintext: string): string  
}
```

4.17.1.9 RsaPair

```
export type RsaPair
```

Initial value:

```
    = {  
    publicKeyPem: string;  
    privateKeyPem: string;  
    }
```

4.17.1.10 SessionEncryptedPayload

```
export type SessionEncryptedPayload
```

Initial value:

```
    = AesGcmPayload & {  
    sessionId: string;  
    }
```

4.17.1.11 SessionTicket

```
export type SessionTicket
```

Initial value:

```
    = {  
  sessionId: string;  
  sessionKey: string;  
}
```

4.18 packages/rpc/src/client.ts File Reference

Variables

- `import { createORPCClient } from "@orpc/client"`
- `import type { AnyRouter, RouterClient } from "@orpc/server"`

4.18.1 Variable Documentation

4.18.1.1 import

```
import { createORPCClient } from "@orpc/client"
```

4.18.1.2 type

```
import type { AnyRouter, RouterClient } from "@orpc/server"
```

4.19 packages/rpc/src/log.ts File Reference

Functions

- function `logDirectory` ()
- function `logPath` (fileName:string)
- function `artifactPath` (fileName:string)
- function export `createSecurityLogger` (fileNames:readonly string[])

Variables

- `import { truncateSync } from "node:fs"`
- `import pino from pino`
- `import pretty from pino pretty`
- export `type SecurityLogActor` = "ttp" | "user" | "server"
- export `type SecurityLogLevel` = "info" | "warn" | "error"
- const `DEFAULT_LOG_DIR` = "../logs"

4.19.1 Function Documentation

4.19.1.1 artifactPath()

```
function artifactPath (  
    fileName:string)
```

4.19.1.2 createSecurityLogger()

```
function export createSecurityLogger (  
    fileNames:readonly string[])
```

4.19.1.3 logDirectory()

```
function logDirectory ()
```

4.19.1.4 logPath()

```
function logPath (  
    fileName:string)
```

4.19.2 Variable Documentation

4.19.2.1 DEFAULT_LOG_DIR

```
const DEFAULT_LOG_DIR = "../../logs"
```

4.19.2.2 import

```
import { truncateSync } from "node:fs"
```

4.19.2.3 pino

```
import pino from pino
```

4.19.2.4 pretty

```
import pretty from pino pretty
```

4.19.2.5 SecurityLogActor

```
export type SecurityLogActor = "ttp" | "user" | "server"
```

4.19.2.6 SecurityLogLevel

```
export type SecurityLogLevel = "info" | "warn" | "error"
```

4.20 packages/rpc/src/server.ts File Reference

Functions

- function export `createRpcFetch` (router:AnyRouter, banner:string)

Variables

- `import type` { AnyRouter } from "@orpc/server"
- `import` { RPCHandler } from "@orpc/server/fetch"

4.20.1 Function Documentation

4.20.1.1 createRpcFetch()

```
function export createRpcFetch (  
    router:AnyRouter,  
    banner:string)
```

4.20.2 Variable Documentation

4.20.2.1 import

```
import { RPCHandler } from "@orpc/server/fetch"
```

4.20.2.2 type

```
import type { AnyRouter } from "@orpc/server"
```


Index

- ActiveSession
 - client-security-state.ts, [14](#)
- activeSession
 - security-flow.ts, [15](#)
- AES_256_KEY_BYTES
 - crypto.ts, [39](#)
- AES_GCM_IV_BYTES
 - crypto.ts, [39](#)
- AES_GCM_TAG_BITS
 - crypto.ts, [39](#)
- aesGcm
 - crypto.ts, [39](#)
- AesGcmPayload
 - types.ts, [41](#)
- api.ts
 - encryptForTtp, [11](#)
 - import, [12](#)
 - loadTtpPublicKey, [11](#)
 - service, [12](#)
 - serviceBaseUrl, [12](#)
 - serviceQuery, [12](#)
 - ttp, [12](#)
 - ttpBaseUrl, [12](#)
 - ttpProtocol, [12](#)
 - ttpQuery, [12](#)
 - type, [13](#)
- apps Directory Reference, [7](#)
- apps/client Directory Reference, [7](#)
- apps/client/src Directory Reference, [8](#)
- apps/client/src/api.ts, [11](#)
- apps/client/src/components Directory Reference, [7](#)
- apps/client/src/components/evidence.tsx, [13](#)
- apps/client/src/components/utls.ts, [13](#)
- apps/client/src/lib Directory Reference, [8](#)
- apps/client/src/lib/client-security-state.ts, [14](#)
- apps/client/src/lib/security-flow.ts, [15](#)
- apps/client/src/lib/useSecurityFlow.ts, [17](#)
- apps/client/src/routes Directory Reference, [8](#)
- apps/client/src/routes/index.tsx, [18](#)
- apps/server Directory Reference, [8](#)
- apps/server/src Directory Reference, [9](#)
- apps/server/src/protocol.ts, [23](#)
- apps/server/src/rpc.ts, [30](#)
- apps/server/src/state.ts, [32](#)
- apps/ttp Directory Reference, [9](#)
- apps/ttp/src Directory Reference, [9](#)
- apps/ttp/src/contract.ts, [20](#)
- apps/ttp/src/protocol.ts, [26](#)
- apps/ttp/src/rpc.ts, [31](#)
- apps/ttp/src/state.ts, [34](#)
- artifact
 - state.ts, [33](#), [34](#)
- artifactPath
 - log.ts, [44](#)
- authenticateServerCertificate
 - protocol.ts, [27](#)
- authenticateUserForServer
 - protocol.ts, [27](#)
- authorityArtifactsSaved
 - protocol.ts, [29](#)
- base64ByteLength
 - index.tsx, [18](#)
- base64ToText
 - index.tsx, [18](#)
- ca
 - state.ts, [34](#)
- CertificateAuthority
 - types.ts, [41](#)
- CertificateAuthorityOptions
 - types.ts, [41](#)
- CertificateClaim
 - protocol.ts, [29](#)
- certificates.ts
 - createCertificateAuthority, [36](#)
 - DAY_MS, [37](#)
 - forge, [37](#)
 - import, [37](#)
 - type, [37](#)
- client-security-state.ts
 - ActiveSession, [14](#)
 - clientSecurityState, [14](#)
 - current, [14](#)
 - import, [14](#)
 - RegisteredUser, [14](#)
- client.ts
 - import, [43](#)
 - type, [43](#)
- CLIENT_STATUS_KEY
 - useSecurityFlow.ts, [18](#)
- clientSecurityState
 - client-security-state.ts, [14](#)
- closeLocalSession
 - protocol.ts, [23](#)
- closeSession
 - protocol.ts, [27](#)
- cn
 - utls.ts, [13](#)

- contract.ts
 - IdentityPublicKeys, 21
 - import, 21
 - RegisterIdentityInput, 21
 - Role, 22
 - ServerAuthenticationInput, 22
 - serverAuthenticationPayload, 21
 - ServerSessionKeyInput, 22
 - UserAuthenticationInput, 22
 - userAuthenticationPayload, 21
 - UserAuthenticationRequest, 22
 - UserAuthRedirectInput, 22
- createCertificateAuthority
 - certificates.ts, 36
- createRogueUserCertificate
 - security-flow.ts, 15
- createRpcFetch
 - server.ts, 45
- createSecurityLogger
 - log.ts, 44
- createSession
 - protocol.ts, 28
- crypto.ts
 - AES_256_KEY_BYTES, 39
 - AES_GCM_IV_BYTES, 39
 - AES_GCM_TAG_BITS, 39
 - aesGcm, 39
 - encodeHybridPayloadBase64, 38
 - encryptRsaOaepSha256, 38
 - forge, 39
 - random, 39
 - rsa, 39
 - RSA_BITS, 39
 - rsaOaepSha256, 38
 - signedPayload, 40
 - type, 40
- current
 - client-security-state.ts, 14
- DAY_MS
 - certificates.ts, 37
- DEFAULT_LOG_DIR
 - log.ts, 44
- DemoFileCurrentResponse
 - security-flow.ts, 16
- DemoFileEmptyResponse
 - protocol.ts, 25
- DemoFileMeta
 - state.ts, 33
- DemoFileResponse
 - protocol.ts, 25
- DemoFileServiceRequest
 - protocol.ts, 25
 - security-flow.ts, 16
- DemoFileServiceResponse
 - protocol.ts, 25
 - security-flow.ts, 16
- DemoFileTransferInput
 - security-flow.ts, 16
- DemoStoredFile
 - state.ts, 33
- encodeHybridPayloadBase64
 - crypto.ts, 38
- EncryptedPayloadView
 - index.tsx, 19
- encryptForTtp
 - api.ts, 11
- encryptRsaOaepSha256
 - crypto.ts, 38
- Evidence
 - evidence.tsx, 13
- evidence.tsx
 - Evidence, 13
- exchangeProtectedServiceData
 - protocol.ts, 23
- fetchServerSessionKey
 - protocol.ts, 24
- FileServiceId
 - index.tsx, 19
- forge
 - certificates.ts, 37
 - crypto.ts, 39
 - protocol.ts, 26, 29
 - security-flow.ts, 16
 - types.ts, 41
- handleFileServiceRequest
 - protocol.ts, 24
- HybridEncryptedPayload
 - types.ts, 41
- IdentityCertificateInput
 - types.ts, 41
- IdentityPublicKeys
 - contract.ts, 21
- IdentityRecord
 - state.ts, 34
- import
 - api.ts, 12
 - certificates.ts, 37
 - client-security-state.ts, 14
 - client.ts, 43
 - contract.ts, 21
 - index.tsx, 19
 - log.ts, 44
 - protocol.ts, 26, 30
 - rpc.ts, 30, 31
 - security-flow.ts, 17
 - server.ts, 45
 - state.ts, 33, 35
 - useSecurityFlow.ts, 18
 - utils.ts, 14
- index.tsx
 - base64ByteLength, 18
 - base64ToText, 18
 - EncryptedPayloadView, 19

- FileServiceId, [19](#)
- import, [19](#)
- isTextFile, [19](#)
- readFileAsBase64, [19](#)
- Route, [19](#)
- SecurityFlowPage, [19](#)
- TransferredFilePreview, [19](#)
- type, [20](#)
- isTextFile
 - index.tsx, [19](#)
- loadRegisteredServer
 - security-flow.ts, [15](#)
- loadTtpPublicKey
 - api.ts, [11](#)
- log
 - state.ts, [32](#), [35](#)
- log.ts
 - artifactPath, [44](#)
 - createSecurityLogger, [44](#)
 - DEFAULT_LOG_DIR, [44](#)
 - import, [44](#)
 - logDirectory, [44](#)
 - logPath, [44](#)
 - pino, [44](#)
 - pretty, [44](#)
 - SecurityLogActor, [44](#)
 - SecurityLogLevel, [44](#)
- logDirectory
 - log.ts, [44](#)
- logger
 - state.ts, [33](#), [35](#)
- logPath
 - log.ts, [44](#)
- packages Directory Reference, [8](#)
- packages/crypto Directory Reference, [7](#)
- packages/crypto/src Directory Reference, [9](#)
- packages/crypto/src/certificates.ts, [36](#)
- packages/crypto/src/crypto.ts, [38](#)
- packages/crypto/src/types.ts, [40](#)
- packages/rpc Directory Reference, [8](#)
- packages/rpc/src Directory Reference, [9](#)
- packages/rpc/src/client.ts, [43](#)
- packages/rpc/src/log.ts, [43](#)
- packages/rpc/src/server.ts, [45](#)
- PendingAuthRecord
 - state.ts, [35](#)
- pendingAuths
 - state.ts, [35](#)
- pino
 - log.ts, [44](#)
- pretty
 - log.ts, [44](#)
- protocol.ts
 - authenticateServerCertificate, [27](#)
 - authenticateUserForServer, [27](#)
 - authorityArtifactsSaved, [29](#)
 - CertificateClaim, [29](#)
 - closeLocalSession, [23](#)
 - closeSession, [27](#)
 - createSession, [28](#)
 - DemoFileEmptyResponse, [25](#)
 - DemoFileResponse, [25](#)
 - DemoFileServiceRequest, [25](#)
 - DemoFileServiceResponse, [25](#)
 - exchangeProtectedServiceData, [23](#)
 - fetchServerSessionKey, [24](#)
 - forge, [26](#), [29](#)
 - handleFileServiceRequest, [24](#)
 - import, [26](#), [30](#)
 - registerIdentity, [28](#)
 - registerProtectedServer, [24](#)
 - requestService, [24](#)
 - requestUserAuthentication, [28](#)
 - serverSessionKey, [29](#)
 - ttp, [26](#)
 - ttpBaseUrl, [26](#)
 - ttpPublicKeyResponse, [29](#)
 - type, [26](#), [30](#)
- random
 - crypto.ts, [39](#)
- readFileAsBase64
 - index.tsx, [19](#)
- readServiceServerStatus
 - state.ts, [32](#)
- registeredIdentities
 - state.ts, [35](#)
- RegisteredServer
 - security-flow.ts, [17](#)
- RegisteredUser
 - client-security-state.ts, [14](#)
- registerIdentity
 - protocol.ts, [28](#)
- RegisterIdentityInput
 - contract.ts, [21](#)
- registerProtectedServer
 - protocol.ts, [24](#)
- registerUserIdentity
 - security-flow.ts, [15](#)
- reject
 - rpc.ts, [30](#), [31](#)
- requestService
 - protocol.ts, [24](#)
- requestUserAuthentication
 - protocol.ts, [28](#)
- resetServiceServerState
 - state.ts, [32](#)
- Role
 - contract.ts, [22](#)
- Route
 - index.tsx, [19](#)
- rpc.ts
 - import, [30](#), [31](#)
 - reject, [30](#), [31](#)
 - ServiceRouter, [30](#)
 - TtpRouter, [31](#)

- type, 31
- rsa
 - crypto.ts, 39
- RSA_BITS
 - crypto.ts, 39
- RsaDecryptor
 - types.ts, 42
- RsaEncryptor
 - types.ts, 42
- rsaOaepSha256
 - crypto.ts, 38
- RsaPair
 - types.ts, 42
- security-flow.ts
 - activeSession, 15
 - createRogueUserCertificate, 15
 - DemoFileCurrentResponse, 16
 - DemoFileServiceRequest, 16
 - DemoFileServiceResponse, 16
 - DemoFileTransferInput, 16
 - forge, 16
 - import, 17
 - loadRegisteredServer, 15
 - RegisteredServer, 17
 - registerUserIdentity, 15
 - securityFlow, 17
 - ServerDemoFileCurrentResponse, 17
- securityFlow
 - security-flow.ts, 17
- SecurityFlowPage
 - index.tsx, 19
- SecurityLogActor
 - log.ts, 44
- SecurityLogLevel
 - log.ts, 44
- server.ts
 - createRpcFetch, 45
 - import, 45
 - type, 45
- ServerAuthenticationInput
 - contract.ts, 22
- serverAuthenticationPayload
 - contract.ts, 21
- ServerDemoFileCurrentResponse
 - security-flow.ts, 17
- serverSessionKey
 - protocol.ts, 29
- ServerSessionKeyInput
 - contract.ts, 22
- service
 - api.ts, 12
- serviceBaseUrl
 - api.ts, 12
- serviceQuery
 - api.ts, 12
- ServiceRouter
 - rpc.ts, 30
- ServiceServerState
 - state.ts, 33
- ServiceServerStatus
 - state.ts, 33
- SessionEncryptedPayload
 - types.ts, 42
- SessionRecord
 - state.ts, 35
- sessions
 - state.ts, 35
- sessionsByRequest
 - state.ts, 36
- SessionTicket
 - types.ts, 42
- signedPayload
 - crypto.ts, 40
- state
 - state.ts, 34
- state.ts
 - artifact, 33, 34
 - ca, 34
 - DemoFileMeta, 33
 - DemoStoredFile, 33
 - IdentityRecord, 34
 - import, 33, 35
 - log, 32, 35
 - logger, 33, 35
 - PendingAuthRecord, 35
 - pendingAuths, 35
 - readServiceServerStatus, 32
 - registeredIdentities, 35
 - resetServiceServerState, 32
 - ServiceServerState, 33
 - ServiceServerStatus, 33
 - SessionRecord, 35
 - sessions, 35
 - sessionsByRequest, 36
 - state, 34
 - type, 36
- TransferredFilePreview
 - index.tsx, 19
- ttp
 - api.ts, 12
 - protocol.ts, 26
- ttpBaseUrl
 - api.ts, 12
 - protocol.ts, 26
- ttpProtocol
 - api.ts, 12
- ttpPublicKeyResponse
 - protocol.ts, 29
- ttpQuery
 - api.ts, 12
- TtpRouter
 - rpc.ts, 31
- type
 - api.ts, 13
 - certificates.ts, 37
 - client.ts, 43

- crypto.ts, [40](#)
- index.tsx, [20](#)
- protocol.ts, [26](#), [30](#)
- rpc.ts, [31](#)
- server.ts, [45](#)
- state.ts, [36](#)
- utils.ts, [14](#)
- types.ts
 - AesGcmPayload, [41](#)
 - CertificateAuthority, [41](#)
 - CertificateAuthorityOptions, [41](#)
 - forge, [41](#)
 - HybridEncryptedPayload, [41](#)
 - IdentityCertificateInput, [41](#)
 - RsaDecryptor, [42](#)
 - RsaEncryptor, [42](#)
 - RsaPair, [42](#)
 - SessionEncryptedPayload, [42](#)
 - SessionTicket, [42](#)
- UserAuthenticationInput
 - contract.ts, [22](#)
- userAuthenticationPayload
 - contract.ts, [21](#)
- UserAuthenticationRequest
 - contract.ts, [22](#)
- UserAuthRedirectInput
 - contract.ts, [22](#)
- useSecurityFlow
 - useSecurityFlow.ts, [18](#)
- useSecurityFlow.ts
 - CLIENT_STATUS_KEY, [18](#)
 - import, [18](#)
 - useSecurityFlow, [18](#)
- utils.ts
 - cn, [13](#)
 - import, [14](#)
 - type, [14](#)